IT biztonság jegyzet

6 Biztonsági tesztelés

6.1 Bevezetés

A szoftvertesztelés és a minőségbiztosítás mindig is a szoftverfejlesztés fontos elemei voltak. Ezek a folyamatok garantálják, hogy a fejlesztés alatt álló szoftver megfeleljen a követelményeknek és a specifikációnak. A nagy nyelvi modellek (LLM-ek) bevezetése a szoftverfejlesztésbe azonban – bár forradalmi változást jelent a meglévő folyamatokban – mélyreható hatással van a kód biztonságára: az LLM-ek által generált kód körülbelül 45%-a nem felel meg a biztonsági teszteknek, és sérülékenységeket vezet be a kódba. Ennek következtében a tesztelés fontossága minden eddiginél nagyobb lett.

Általánosságban a szoftvertesztelés folyamata 5 fő szakaszból áll:

1. Teszttervezés: ebben a szakaszban meghatározzák a célokat, a hatókört, az erőforrásokat és az ütemtervet, valamint a lezárási kritériumokat (mikor tekinthető a teszt sikeresnek, mikor sikertelennek).
2. Tesztkialakítás: ebben a szakaszban absztrakt teszteseteket – például az agilis tesztelési módszertanok felhasználói történeteit – alkalmaznak, és meghatározzák a tesztelési feltételeket.
3. Implementáció: ebben a fázisban a megtervezett teszteseteket végrehajtják.
4. Kivitelezés: ebben a fázisban a végrehajtott teszteseteket futtatják, és a futtatásukat naplózzák, valamint figyelemmel kísérik a releváns részleteket.
5. Értékelés: ebben a fázisban a futtatott tesztesetek eredményeit áttekintik és a kilépési kritériumok alapján értékelik.

Ezek a fázisok mindig megmaradnak, függetlenül attól, hogy a tesztelés hogyan van szervezve a fejlesztési módszertanban. A hagyományos szoftverfejlesztési módszertanokban, mint például a vízesés- és a V-modell, ezeket a fejlesztési folyamat külön fázisaként hajtják végre. Az agilis tesztelésben (pl. test-driven development, behavior-driven development vagy acceptance test-driven development) ezek beágyazódnak a fejlesztési folyamatba: a teszttervezés, a tesztkialakítás és az implementáció a funkcionális megvalósítás előzményei.

A biztonsági tesztelés egy olyan tesztelési módszer, amely független a többi szoftvertesztelési megközelítéstől, például a funkcionális és nem funkcionális teszteléstől. Célja annak ellenőrzése, hogy a biztonsági követelmények teljesülnek-e. E cél elérése érdekében a biztonsági tesztelés két fő tevékenységet foglal magában: a sérülékenységek azonosítását és a biztonsági funkciók működésének biztosítását. Előfordulhat, hogy a rendszer tervezett biztonsági funkcionalitása (amelyet a követelmények és a specifikáció tartalmaz) és a megvalósított biztonsági funkcionalitás nem egyezik meg. A biztonsági tesztelés feltárhatja a hiányzó vagy helytelen biztonsági funkcionalitást, valamint a megvalósított biztonsági funkcionalitás nem szándékolt mellékhatásait.

A sikeres biztonsági tesztelés végrehajtása három fő okból jelent kihívást:

1. A megfelelő gondolkodásmód: A biztonsági teszteléshez a tesztelőknek „a kereteken kívül” kell gondolkodniuk, hasonlóan a támadókhoz. Erre azért van szükség, mert a biztonsági teszteléshez használt automatizált eszközök nem gondolkodnak kreatívan.
2. A tárgy megértése: A tesztelőknek meg kell ismerkedniük a tesztelni kívánt alkalmazással/rendszerrel. Ehhez az alkalmazás/rendszer pontos dokumentációjának beszerzése, tanulmányozása és megértése szükséges, ami javítja az összes teszt hatékonyságát.
3. Jó mérőszámok: Mint minden tesztelési megközelítésnél, a biztonsági tesztelésnél is szükség van mérőszámokra az ellenőrizhetőség érdekében. Jó mérőszámok segítségével a teszt eredményei felhasználhatók annak értékelésére, hogy szükség van-e további oktatásra vagy képzésre, hogy a fejlesztők egyértelműen megértik-e egy adott biztonsági mechanizmust, és hogy csökken-e a feltárt biztonsági problémák összesített száma.

Ebben a fejezetben különböző biztonsági tesztelési technikákat dolgozunk fel. Először azt tárgyaljuk, hogyan lehet statikus és dinamikus elemzési technikák segítségével sérülékenységeket találni a komponensekben. Ezután a biztonsági tesztelés CI/CD folyamatba való integrálását tárgyaljuk. Végül a működő rendszerekben végzett biztonsági tesztelést vizsgáljuk.

6.2 Sérülékenység keresés komponensekben

A biztonsági tesztelés programelemzési technikákra támaszkodik a szoftverkomponensekben rejlő sérülékenységek felkutatásához. A programelemzési technikák e tekintetben két kategóriába sorolhatók: statikus programelemzés és dinamikus programelemzés. Ebben a fejezetben ezeket a megközelítéseket tárgyaljuk, beleértve azok előnyeit és hátrányait is.

Mint később látni fogjuk, a statikus és a dinamikus programelemzési technikák egymást kiegészítő erősségekkel és gyengeségekkel rendelkeznek. Ezért a biztonsági tesztelési folyamatnak mindkettőt be kell építenie, hogy a két megközelítés előnyeit egyaránt kihasználhassa.

6.2.1 Statikus programelemzés a biztonsági rések felderítéséhez

A statikus programelemzési technikák bemeneti adatként egy utasításkészletet vesznek figyelembe. Ezek az utasítások lehetnek forráskód vagy gépi kód, de köztes ábrázolások (intermediate representation) és köztes nyelvek (intermediate language) is. Az utóbbi kettő a szoftver absztrakt ábrázolása, amely hídként szolgál a forráskód és a gépi kód között. A közbenső ábrázolások magasabb szintűek, mint a gépi kód, de absztrakt regisztereikkel és alapvető műveleteikkel még mindig alacsony szintűek. A köztes nyelvek hasonlóak ahhoz, ahogyan a fordítók feldolgozzák a forráskódot: alacsonyabb szintűek, mint a forráskód, de még mindig magas szintűek, komplex utasításokkal, például ciklusokkal.

A statikus programelemzési technikák csupán értelmezik az utasításokat, de nem hajtják végre azokat. Előnyük, hogy skálázhatók, és már a fejlesztési folyamat korai szakaszában alkalmazhatók (valójában már abban a pillanatban, amikor a forráskód rendelkezésre áll). Bizonyos statikus programelemzési technikák manuális végrehajtása azonban fárasztó. Általában automatizáljuk ezeket a technikákat, elfogadva azt a limitációt, hogy az eszközök csak előre meghatározott sérülékenységi mintákat keresnek. Mivel nincs rendelkezésre álló futásidejű információ, a statikus programelemzési technikáknál gyakoriak a téves riasztások – olyan kódot jelölnek meg sebezhetőnek, amely valójában nem jelent biztonsági kockázatot. Az ilyen riasztások felkutatása és kiszűrése a tesztelő feladata.

Néhány példa a statikus programelemzési technikákra:

- Code review: az utasítások végigolvasása azok futtatása nélkül, a sérülékenységek felkutatása céljából.
- Taint analízis: egy magas kockázatú adat a szoftverben végigjárt lehetséges végrehajtási útvonalainak elemzése, a biztonsági ellenőrzések megkerülések felkutatása céljából.
- Szimbolikus végrehajtás: a szoftverkomponensek végrehajtásának szimulálása egy absztrakt környezetben.

6.2.1.1 Manuális code review

A kézi code review során szakértők sorról sorra átolvasják a kódot a sérülékenységek felderítése érdekében. Ez a folyamat a komponensek fenyegetési modelljének elkészítésével, de legalábbis a támadási felület feltérképezésével kezdődik. A manuális code review a következő gyenge pontokat keresi:

- Úgynevezett veszélyes konstrukciók: olyan függvények vagy tervezési minták használata, amelyekről ismert, hogy sérülékenységeket okoznak.
- Injection lehetőségek, beleértve az SQL-injectiont, a shell-parancsok végrehajtását, az XML-injectiont, stb.
- Az egész számok kezelése, előfordulhat-e túlcsordulás és alulcsordulás, előjeles/előjel nélküli eltérések, csonkolás.
- Hiányzó vagy nem megfelelő bemeneti érvényesítés (ami megnöveli a támadási felületet) és/vagy kimeneti szűrés (ami más komponenseket is veszélybe sodorhat).
- Nyelvspecifikus problémák.

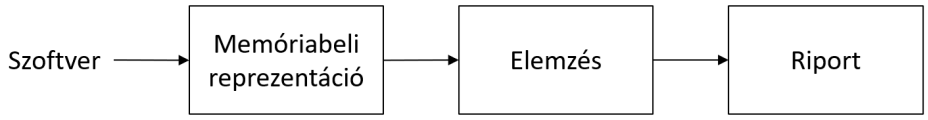
A szoftvergyengeségek átfogó listája megtalálható a Common Weakness Enumeration (CWE) adatbázisban.

A manuális code review előnye az emberi kreativitás. Jelenleg egyetlen eszköz sem képes utánozni azt a sajátos, „kereteken kívüli" gondolkodásmódot, amelyet az emberi szakértők hoznak a code review folyamatába. A manuális code review azonban fárasztó folyamat, és számos területen szakértelmet igényel: a tesztelőnek ismernie kell az architektúrát és a megvalósítási technikákat, emellett jártasnak kell lennie a biztonsági kérdésekben is, stb.

6.2.1.2 Automatizált code review

A kódellenőrzést végző szakértők munkájának támogatására automatizált code review eszközöket fejlesztettek ki. Ezek az eszközök – hasonlóan az emberi szakértőkhöz – elemzik az utasításokat, és jelentést készítenek a kódban talált lehetséges problémákról. A biztonsági tesztelőknek csupán át kell tekinteniük az eszközök által generált jelentéseket. Az iparban ezeket az eszközöket statikus alkalmazásbiztonsági tesztelő (static application security testing, SAST) eszközöknek is nevezik.

Ezeknek az eszközöknek az előnye, hogy használatukhoz nincs szükség szakértelemre. Hátrányuk viszont, hogy csak azokat a problémákat jelzik, amelyekre be vannak programozva. Ezért az, hogy nem jelenik meg figyelmeztetés, nem jelenti azt, hogy a kód biztonságos lenne. Megjegyzés: Ritkán fordul elő, hogy a SAST-eszközök ne jelentenek problémákat. Általában éppen ellenkezőleg: bármilyen ésszerű méretű kódbázis esetén is hatalmas mennyiségű jelentést generálnak.



SAST eszközök magas-szintű működése

A szoftverek elemzéséhez a SAST-eszközök először a bemenetet memóriában tárolt belső reprezentációra alakítják át. Hogy ez az ábrázolás pontosan milyen, az az eszköztől függ: alapulhat statikus programelemzési koncepciókon, vagy lehet egy saját fejlesztésű ábrázolási formátum is.

Az eszközök ezután statikus programelemzési technikákat alkalmaznak a memóriában tárolt ábrázoláson. A megvalósított elemzési technikák általában olyan mintákat keresnek, amelyekről ismert, hogy problémákat okoznak, hasonlóan ahhoz, amit a manuális code review is keres.

Végül a SAST eszközök jelentést készítenek az elemzések eredményeiről. A jelentés hibákat tartalmazhat:

- Hamis pozitív riasztás: a nem sebezhető kódot sebezhetőként jelöli meg.
- Hamis negatív riasztás: a sebezhető kódot nem jelöli meg.

Ezért a SAST eszközök eredményeit más biztonsági tesztelési technikákkal is ellenőrizni kell.

6.2.2 Dinamikus programelemzés a biztonsági rések felkutatásához

A dinamikus programelemzés a szoftvert valós körülmények között, futás közben elemzi. Ennek köszönhetően hozzáfér a futásidejű információkhoz, ami pontosabb eredményeket biztosít. Ezen felül a dinamikus elemzés által jelzett biztonsági rések biztosan jelen vannak a szoftverben.

Hátránya viszont, hogy a dinamikus programelemzés elvégzéséhez programinstrumentálásra lehet szükség, vagyis a szoftver szándékos módosítására annak érdekében, hogy futását megfelelően lehessen figyelemmel kísérni. A tesztelt szoftvertől függően ez nem feltétlen triviális lépés, például a forráskód nélküli bináris fájlok instrumentálása köztudottan nagy kihívást jelent. Ráadásul a dinamikus programelemzési technikák alacsony kódlefedettséget biztosítanak: a szoftverkomponenseknek csak egy kis részét tudják tesztelni. A dinamikus elemzési technikák csak a megfigyelt viselkedés alapján tudnak következtetéseket levonni: ha a tesztelés során a sérülékeny végrehajtási útvonalakat nem érintjük (pl. a tesztkészlet hiányossága miatt), a dinamikus elemzés nem fogja észlelni a sérülékenységet.

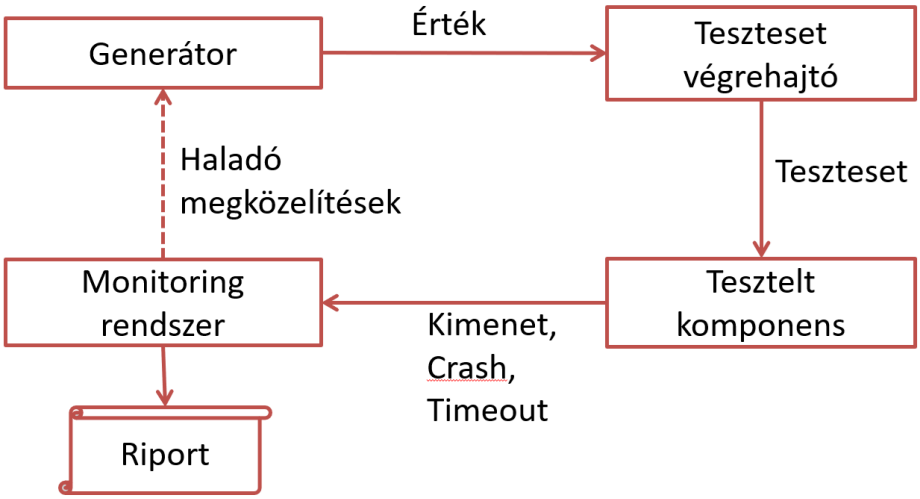
Néhány példa a dinamikus programelemzési technikákra:

- Debuggolás és trace generálás: egy komponens konkrét végrehajtási útjának követése egy adott bemeneti adatsor esetén.
- Fuzzing: a szoftverkomponens viselkedésének figyelemmel kísérése, miközben azt (félig) véletlenszerűen generált bemeneti adatokkal bombázzuk.
- Dinamikus taint analízis: egy magas kockázatú adat konkrét végrehajtási útjának követése a szoftverben, a biztonsági rések felkutatása céljából.

Az ezeket az elemzéseket implementáló eszközöket Dynamic Application Security Testing (DAST) toolként is szokás nevezni.

6.2.2.1 Fuzzing

A fuzzing egy dinamikus programelemzési technika, amelyet széles körben alkalmaznak a biztonsági tesztelés során. A technika alapgondolata az, hogy a tesztelt szoftverkomponenst véletlenszerűen generált bemeneti adatokkal bombázzák, és figyelik a viselkedését, hogy hibákat vagy másfajta rendellenességeket észleljenek. Ez a tesztelési módszer jól alkalmazható puffer túlcsordulások, hiányzó bemeneti érvényesítések, C/C++ memóriakorruptságok, valamint egész számokkal kapcsolatos problémák felderítésére. Alkalmazásának fő kihívása az, hogyan lehet eljutni a „mély” kódhoz: a szigorú input validációval védett kódhoz.



A fuzzing magasszintű workflow-ja

A fuzzer általában három fő komponensből áll:

- Generátor: Ez a komponens egy megadott stratégia alapján generál véletlenszerű bemeneti adatokat (pl. tisztán véletlenszerű bemenet, a specifikációnak megfelelő bemenet, stb.).
- Teszteset végrehajtó: Ez a komponens a generált véletlenszerű értéket veszi át, és azt végrehajtható tesztesetté alakítja. Például létrehozhat egy helyes HTTP-kérés, amelynek query paramétere a generált véletlenszerű értékre van beállítva. Ezenkívül végrehajtja a tesztesetet, ezáltal a generált véletlenszerű értéket átadva a tesztelt szoftverkomponensnek.
- Felügyeleti rendszer: Ez a komponens figyeli a tesztelt szoftverkomponens kimenetét. Általában összeomlásokat, időtúllépéseket és egyéb hibás viselkedéseket keres.

6.2.2.2 Manuális és automatizált dinamikus biztonsági tesztelés

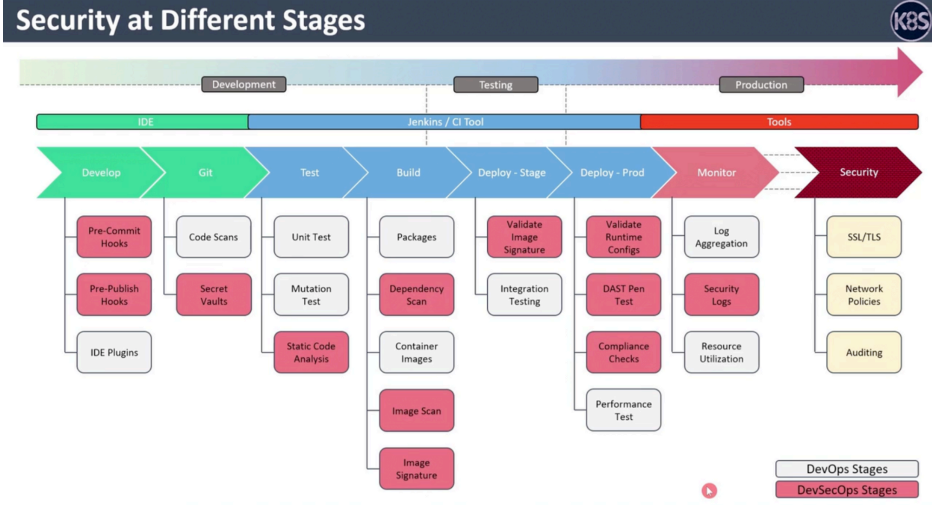
A dinamikus biztonsági tesztelés – bár eszközökkel jelentősen támogatott – nem feltétlenül teljesen automatizált folyamat. Éppen ellenkezőleg: a tesztelési folyamatnak magában kell foglalnia a kézi értékelést is, amely kiterjeszti a tesztteret és azonosítja a problémás részeket. A futásidejű elemzés tekintetében a manuális és az automatizált megközelítések egymást kiegészítő erősségekkel és gyengeségekkel rendelkeznek. Ilyen esetekben a különböző megközelítések között szinergia érhető el.

Az automatizált futásidejű elemzés a tesztvektorok szisztematikus és automatizált generálása. Az eszközök adaptálása és összekapcsolása kezdeti befektetést igényel, de ezt a befektetést csak egyszer kell megtenni, ami számos teszt végrehajtását teszi lehetővé. A tesztkörnyezet felállítása után az automatizált futásidejű elemzés több tízezer tesztet futtathat, ami a lefedettség jelentős növekedéséhez vezet. Ez a megközelítés a tipikus hibák többségét képes megtalálni, de gyakran nem veszi figyelembe a szélsőséges eseteket és a speciális problémákat.

A manuális futásidejű elemzés pótolhatja az automatizált futásidejű elemzés ezen hiányosságát. Ez magában foglalja a saját fejlesztésű tesztesetek egyedi kidolgozását, ami jelentős erőfeszítést igényel az egyes tesztesetekhez szükséges egyedi kód megírásához. Előnye, hogy szakértelem alapján képes feltárni a szoftverkomponens mélyén rejlő speciális problémákat. Az ehhez szükséges erőfeszítés azonban csak néhány tucat ilyen teszteset kidolgozását teszi lehetővé, amelyek a tesztelt kód legkockázatosabb részeire koncentrálnak.

6.3 A biztonsági tesztelés beépítése a CI/CD-folyamatba

A modern szoftverfejlesztés alapja, hogy a fejlesztők egymást gyorsan követő lépésekben töltenek fel új és módosított kódot a repókba. Ennek következtében a commitok közötti idő annyira lerövidült, hogy a biztonsági tesztelési folyamat kézi elindítása hosszú távon már nem fenntartható. E kihívás leküzdése érdekében a SAST- és DAST-eszközöket egyre inkább beépítik a CI/CD-folyamatokba. Ezen felül új biztonsági tesztelési technikákat is kifejlesztettek a sérülékenységek felderítésére.



Integrating security testing into a CI/CD pipeline

A függőségi vizsgálatok (dependency scan), más néven szoftverösszetétel-elemzés (SCA), a szoftverellátási láncra összpontosítanak. Konkrétan a lefordított kód függőségi fáját vizsgálják át, ismert sérülékenységeket keresve. Az ismert sérülékenységek felderítéséhez a CVE-adatbázisra, a függőségek nyomon követéséhez pedig a Software Bill of Materials-ra (SBOM) támaszkodnak, például az SPDX vagy az OWASP CycloneDX formátumú fájlokra.

Ha a szoftverkomponensek konténer image-ekben érhetők el, ezeket az image-eket is át lehet vizsgálni sérülékenységek után. Az ilyen típusú ellenőrzést végző eszközök általában elavult könyvtárakat és operációs rendszereket, valamint helytelen/biztonságtalan konfigurációkat keresnek. Vannak olyan eszközök is, amelyek adott kritériumoknak való megfelelést ellenőrzik.

6.4 Élő rendszerek biztonsági tesztelése

Az alkalmazások és az informatikai rendszerek üzembe helyezése és üzemeltetése után is léteznek olyan biztonsági tesztelési technikák, amelyekkel fel lehet tárni a rendszerekben rejlő sérülékenységeket. A nagyobb vállalatok bug bounty programokat indítanak az etikus hackerek bevonására, amelyek keretében pénzjutalmat ígérnek a megerősített súlyos sérülékenységek felfedezéséért. A red teaming egy engedélyezett, szimulált ellenséges támadás a rendszer ellen, amely a valós támadásokat végrehajtó kifinomult támadók ellen teszteli a rendszer biztonsági hatékonyságát. A penetrációs tesztelés (rövidítve pentesting) egy adott rendszer, alkalmazás vagy hálózati szegmens célzott értékelése a kihasználható sérülékenységek felderítése céljából.

6.4.1 Penetrációs tesztelés

A penetrációs tesztelés egy teszt- vagy production környezetben működő rendszerre összpontosít. Felépítése hasonló egy valódi támadáshoz: automatizált eszközök és manuális technikák egyaránt felhasználásra kerülnek a támadók által kihasználható gyenge pontok feltárása érdekében. A penetrációs tesztelés célja a tesztelt rendszer támadási felületének felmérése az elérhető interfészek szisztematikus vizsgálatával és a feltárt sérülékenységek kihasználásával.

Számos szabvány létezik, amely részletesen leírja a penetrációs tesztelés lépéseit és fázisait. Általánosságban a következő 4 fázist különböztethetjük meg:

1. Tervezés: Ebben a fázisban az ügyfél és a penetrációs tesztelők egyeztetik a fontos keretfeltételeket és határokat. Ez az egyeztetés többek között magában foglalja a teszt hatálya alá tartozó rendszerkomponensek listáját, a teszt időtartamát és ütemezését. Végül szerződést írnak alá, amely felhatalmazza a penetrációs tesztelőket az értékelés elvégzésére.
2. Felfedezés: A penetrációs tesztelők automatizált és manuális eszközöket használnak az összes elérhető interfész felfedezésére és felsorolására, amelyet a sérülékenységi elemzés követ. A sérülékenységi elemzéshez sérülékenységi szkennerek használhatók, és a nyilvános sérülékenységi adatbázisok (pl. CVE) is felhasználhatók az ismert komponensek esetében.
3. Támadás: Ebben a fázisban a azonosított interfészeket támadási kísérletekkel tesztelik. Sikeres kihasználás esetén a tesztelők további információkat keresnek a rendszerben az API-król és a szolgáltatásokról, és új komponenseket találnak. Ezáltal a rendszer támadási felülete kibővül, ami visszavezet a Felfedezés fázisba.
4. Jelentés: A többi fázissal párhuzamosan a tesztelők dokumentálják az összes megállapításukat és a sérülékenységek becsült súlyosságát.

Impresszum | Copyright © 2018 BME Villamosmérnöki és Informatikai Kar
1117 Budapest, Magyar tudósok körútja 2. | +36 1 463 3581 | moodle@vik.bme.hu